



FEU INSTITUTE OF TECHNOLOGY

IT0093

(MOBILE APPLICATION DEVELOPMENT 2)

Machine Problem

1

**“Campus Club Connect” — Welcome Screen, Navigation, and
User Input**

Student Name:	Soriano, John Ronen E.
Section:	BSIT:WMA/TW33
Professor:	Prof. Von Magbitang
Github Repository:	https://github.com/Zcommand/soriano_ios_appdev/tree/main/TSA/ClubConnect



1. Coverage

This Machine Problem (MP) is a graded, individually-submitted deliverable that consolidates the material from Week 1 (Topic 1.1 - Getting Started with iOS Development; Topic 1.2 - Storyboard and User Interface Design) and Week 2 (Topic 2.1 - Navigation Controller and Screen Navigation; Topic 2.2 - User Input and Interface Components) of the Laboratory Manual.

Unlike the guided Campus Companion iOS activity built in class, this MP asks you to build a separate, independently-scoped application, Campus Club Connect, from a written specification. The underlying UIKit mechanisms are the same ones practiced in the Week 1-2 labs; only the screen names, labels, and input values differ. Copying the Campus Companion project files and renaming classes does not satisfy this requirement – the project must be created from a new Xcode template following the steps below.

Week / Topic	Concept	Where it appears in this Machine Problem
1.1 - Getting Started	Xcode project creation, Storyboard/UIKit template, Simulator, Git init	Part A, Steps 1-2
1.2 - Storyboard & UI Design	Object Library, Auto Layout constraints, IBOutlet	Part A, Steps 3-5
2.1 - Navigation Controller	Embed In > Navigation Controller, second view controller subclass, manual identified segue, IBAction, performSegue(withIdentifier:sender:)	Part B, Steps 6-8
2.2 - User Input & Data Passing	UITextField, UISwitch, UISegmentedControl, IBOutlet for each, prepare(for:sender:), guard, Optional handling	Part C, Steps 9-11

2. CLO Alignment

- CLO 1 – Apply Swift programming concepts and UIKit components (UIViewController, UILabel, UIImageView, UITextField, UISwitch, UISegmentedControl, IBOutlet, IBAction).
- CLO 2 – Design a Storyboard-based iOS application that incorporates a multi-screen user interface satisfying a written specification.
- CLO 3 – Follow professional development practices: version control from project creation, descriptive naming, defensive handling of optional user input, and a written reflection.

3. Scenario

Campus Club Connect is a lightweight companion app that helps students discover and register interest in a campus organization. A student opens the app, sees a welcome screen introducing the app, enters their name, indicates whether they want meeting reminders, and selects whether they are joining as a Member or an Officer. Tapping a button takes them to a confirmation screen summarizing their registration.



FEU INSTITUTE OF TECHNOLOGY

You are free to choose your own club name, color scheme, and image placeholder, provided every functional requirement in Section 4 is met.

4. Expected Output / Outcomes

The two mockups below illustrate the target outcome described in Section 3 and the requirements in Section 4 – they are a visual reference, not a required color scheme or layout. Your own title, tagline, colors, and logo placeholder may differ, as noted above.

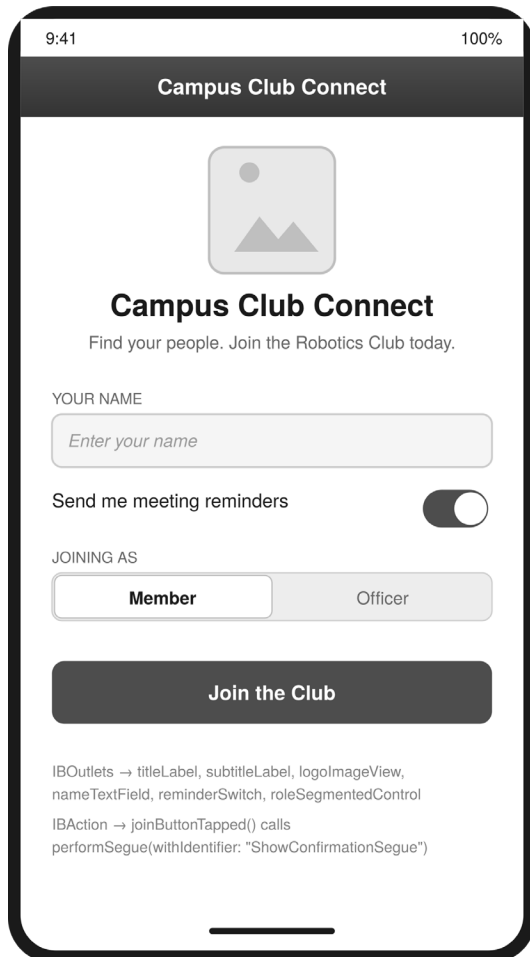


Figure 1. Welcome screen — title, subtitle, logo placeholder, name field, reminder switch, role segmented control, and Join button.

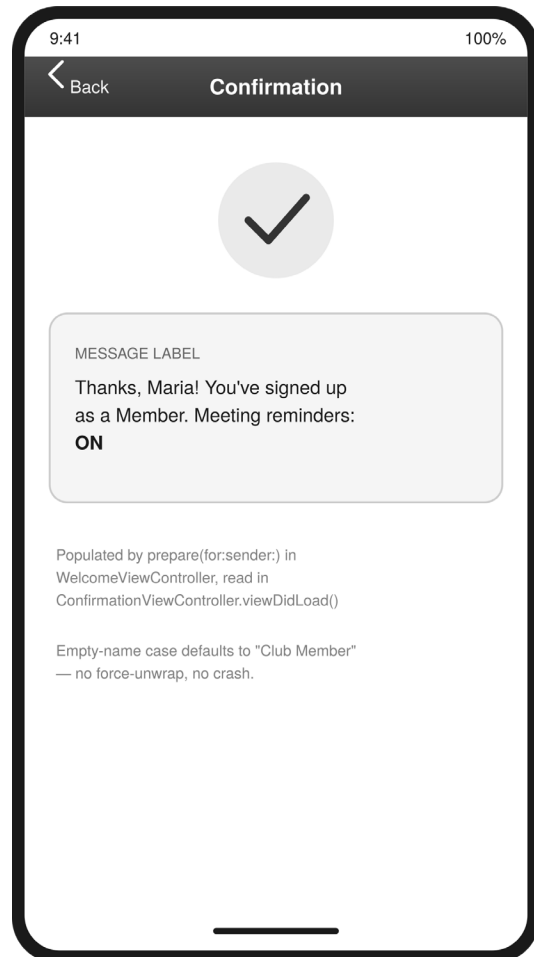


Figure 2. Confirmation screen — auto-supplied Back button and a message label built from the values passed in `prepare(for:sender:)`.

The diagram below traces how the three captured input values move from the welcome screen to the confirmation screen through the manual segue and `prepare(for:sender:)` – the mechanism required in Part C.

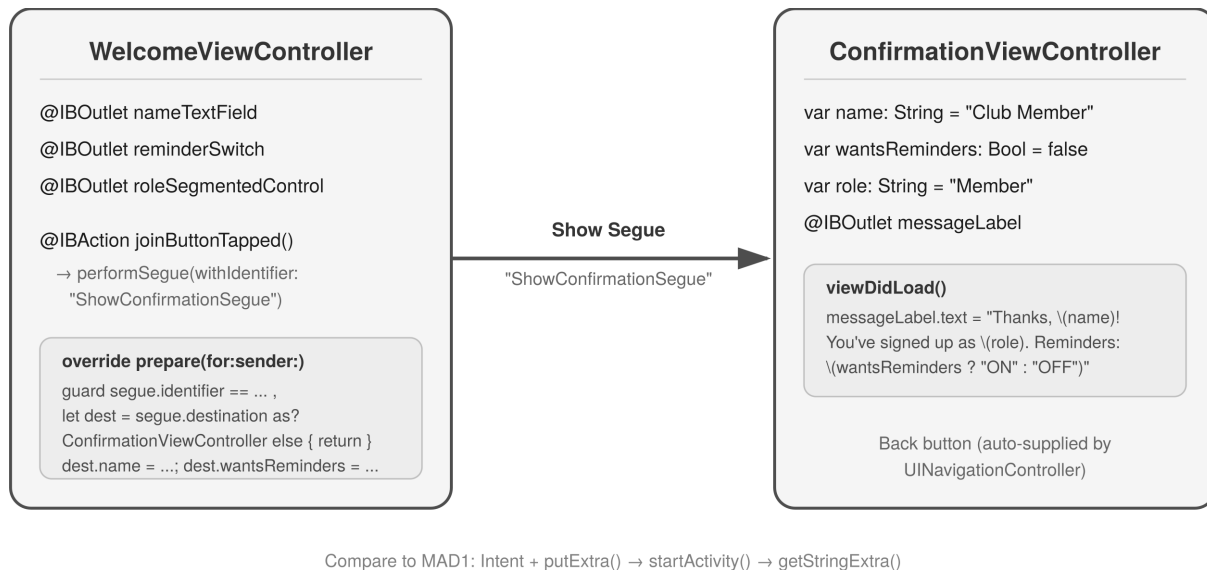


Figure 3. Navigation and data-passing flow between WelcomeViewController and ConfirmationViewController.

5. Functional Requirements

Part A — Project Setup and Welcome Screen (Week 1)

- Create a new Xcode project using the App template with Interface: Storyboard, Language: Swift, Use Core Data: unchecked, Include Tests: unchecked. Name the product ClubConnect.
- Initialize a Git repository at project creation (Source Control > Create Git Repositories...).
- On the initial scene (rename the class to WelcomeViewController), add and constrain, relative to the Safe Area:
 - an app title label (e.g., “Campus Club Connect”), bold, ≥ 24pt
 - a subtitle label describing the club or app tagline
 - an image view (Aspect Fit, fixed width/height) as a placeholder for a future club logo
- Connect the title label to an IBOutlet and set its text programmatically inside viewDidLoad() rather than leaving it hard-coded on the canvas.
- Verify the layout on at least two simulator devices of different screen sizes (for example, iPhone SE (3rd generation) and iPhone 15 Pro Max) with no clipping or overlap.

Part B — Navigation Controller and a Second Screen (Week 2, Topic 2.1)

- Embed the welcome scene inside a UINavigationController (Editor > Embed In > Navigation Controller) and set its navigation title.
- Add a button to the welcome screen (e.g., “Join the Club”).



FEU INSTITUTE OF TECHNOLOGY

- Create a second scene with its own UIViewController subclass, ConfirmationViewController, containing at minimum a message label.
- Draw a manual segue from the welcome scene's view controller icon (not directly from the button) to the confirmation scene, of kind Show, and give it a descriptive identifier such as ShowConfirmationSegue.
- Connect the button to an IBAction that calls performSegue(withIdentifier:sender:) using that identifier.
- Run the app and confirm the confirmation screen appears with a working, automatically supplied Back button.

Part C — User Input and Data Passing (Week 2, Topic 2.2)

- Add three input components to the welcome screen, positioned and constrained consistently with the elements added in Part A:
 - a UITextField for the student's name (with a placeholder)
 - a UISwitch for a meeting-reminder preference
 - a UISegmentedControl with two segments, e.g., “Member” and “Officer”
- Connect all three to IBOutlets in WelcomeViewController.
- Add matching properties to ConfirmationViewController (with sensible default values) and an IBOutlet for the message label.
- Override prepare(for:sender:) in WelcomeViewController to: confirm the segue identifier and cast segue.destination to ConfirmationViewController using a guard statement; read the text field's value safely (never force-unwrap), substituting a default such as “Club Member” if it is empty; read the switch's isOn value; read the segmented control's selectedIndex and map it to the corresponding role string; and assign all three to the destination's properties.
- In ConfirmationViewController's viewDidLoad(), assemble the three values into a single personalized message and display it in the message label.
- Call resignFirstResponder() on the text field before the segue executes.
- Test both the populated case and the empty-text-field case; the app must not crash in either case.

6. Deliverables

- The complete ClubConnect Xcode project folder (including the .git history), zipped.
- A screenshot of the welcome screen with sample values entered.
- A screenshot of the confirmation screen showing the resulting message.
- A screenshot demonstrating the layout on two different simulator device sizes (Part A, Step 5).
- A brief written reflection (3-5 sentences) comparing prepare(for:sender:) and the manual segue mechanism used here to the Intent-based navigation and putExtra()/getStringExtra() pattern from Mobile Application Development 1.

7. Grading Rubric (100 points)

Criterion (100 pts)	Excellent	Good	Fair	Poor
---------------------	-----------	------	------	------



FEU INSTITUTE OF TECHNOLOGY

1. Application Functionality & Execution (30 pts)	27–30 – Runs successfully with all required features working and no major errors.	23–26 – Runs successfully with minor errors or issues.	18–22 – Runs with several incomplete features or noticeable errors.	12–17 – Does not run properly or has major non-functional features.
2. User Interface & User Experience (20 pts)	18–20 – Clear, consistent, and user-friendly interface with proper navigation.	15–17 – Generally usable with minor design or navigation issues.	12–14 – Usable but has noticeable layout or navigation problems.	8–11 – Difficult to use or poorly organized interface.
3. Code Quality, Documentation & Understanding (20 pts)	18–20 – Well-organized code with meaningful comments; student clearly explains the implementation.	15–17 – Organized code with adequate comments; student explains most of the implementation.	12–14 – Limited comments or organization; student explains only some of the code.	8–11 – Poorly organized or undocumented code; student cannot adequately explain the implementation.
4. Completeness & Requirements Compliance (15 pts)	14–15 – All required features and components are complete and follow specifications.	11–13 – Most requirements are complete with minor omissions.	8–10 – Several requirements are incomplete or missing.	5–7 – Many requirements are missing or not properly implemented.
5. Submission, Delivery & Compliance (15 pts)	14–15 – Complete submission is submitted on or before the deadline and follows all instructions.	11–13 – Complete submission is within 1 day late with minor submission issues.	8–10 – Submission is 2–3 days late and/or has missing components.	5–7 – Submission is more than 3 days late or substantially incomplete.

Code Documentation & Understanding Requirement

Students must include meaningful comments/documentation for important parts of their source code. Comments should explain the purpose or logic of the code, not simply restate the code. The instructor may ask students to explain selected portions of their code to verify their understanding of the submitted implementation.

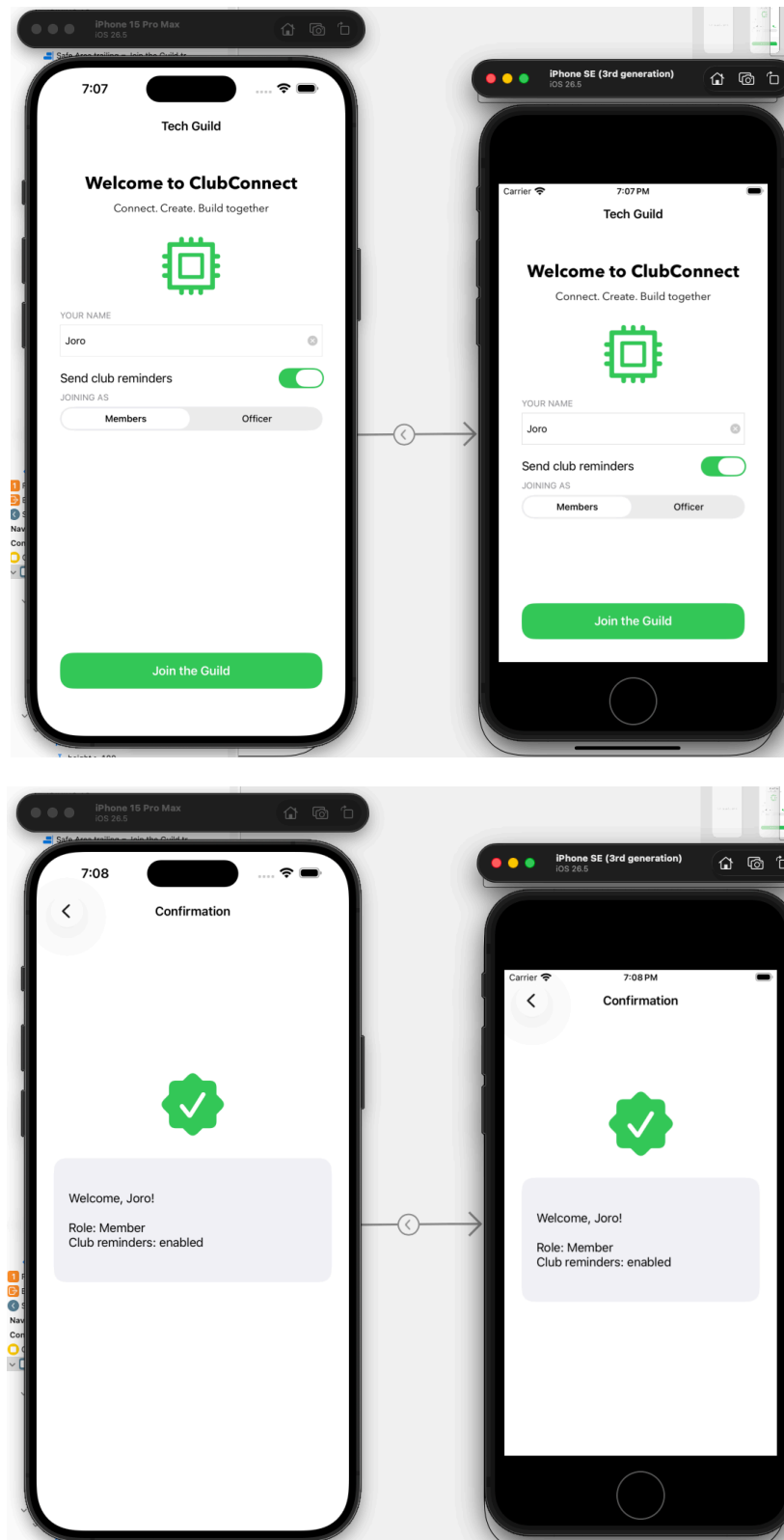
Code Verification Rule

Code Verification: Students may be asked to explain or demonstrate selected portions of their submitted source code. Failure to reasonably explain significant portions of the implementation may result in a lower rating under Code Quality, Documentation & Understanding.



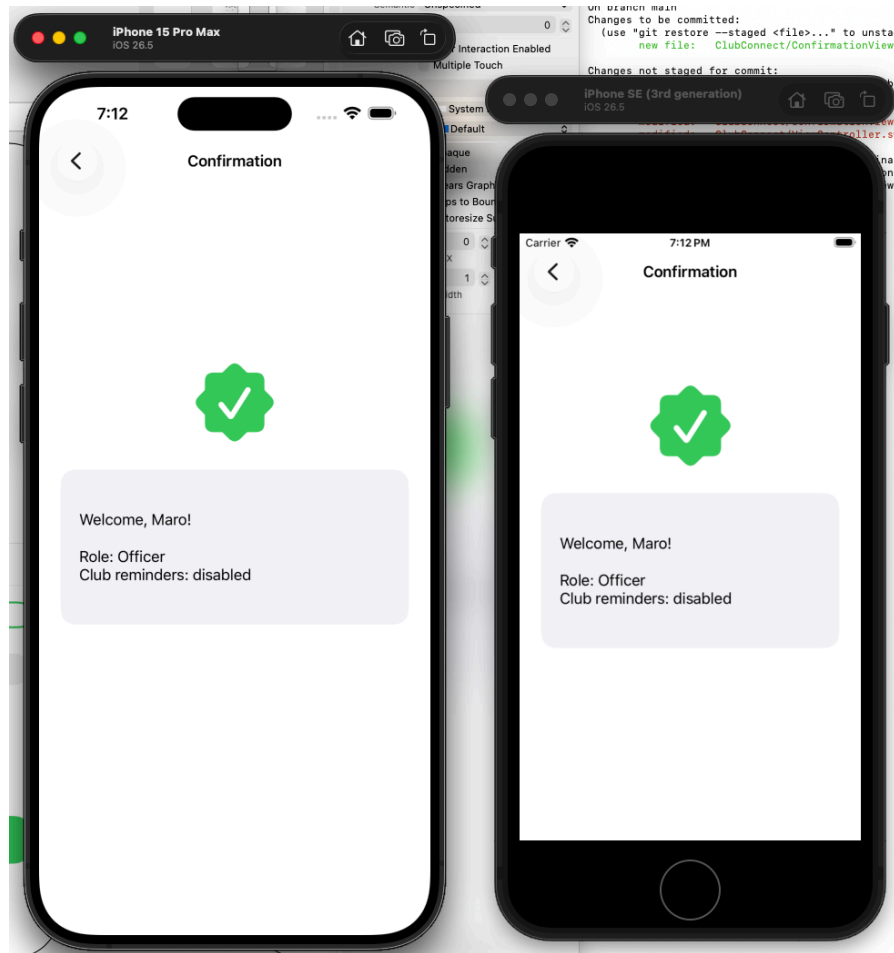
FEU INSTITUTE OF TECHNOLOGY

SAMPLE OUTPUT:





FEU INSTITUTE OF TECHNOLOGY





8. Submission Instructions

- Submit the zipped project and screenshots through the course LMS assignment link before the posted deadline.
- File name format: LastName_FirstName_MP1_ClubConnect.zip.
- Late submissions are subject to the course's standard late-penalty policy.

9. Academic Integrity and AI Use

You may use AI tools the same way modeled in the Week 1-2 AI-Assisted Learning sections: to clarify a concept (for example, why `UITextField.text` is Optional, or how `UINavigationController` compares to the Android back stack), or to help interpret an Xcode error message. You must be able to explain every line of your own submission, and any AI-suggested code must be verified by running it in your own project rather than copied without understanding. Submitting the completed Campus Companion in-class project (renamed) in place of an independently-built Campus Club Connect project is considered a failure to meet the assignment requirements, not an academic integrity shortcut – but it will not receive credit for Parts A-C.

10. What This Prepares You For

The navigation and data-passing pattern practiced here – a Navigation Controller, a manual segue, and `prepare(for:sender:)` – is the same pattern Week 3 reuses when navigating from a `UITableView` list of items to a detail screen for a selected row. Completing this MP confirms you are ready to combine navigation with dynamic, data-driven lists starting next week.



AI DECLARATION:

I used AI as a learning assistant while developing this project. It guided me through setting up the storyboard layout, Auto Layout constraints, outlets/actions, segues, and passing the entered name, reminder setting, and selected role to the confirmation screen. I reviewed the suggestions, applied the changes in Xcode myself, tested the app on different simulator devices, and verified that the final output worked correctly.

Development [AppDev2]

Share

assist me to accomplish the attached context for a certain school work. begin by summarizing the crucial information that I should know and walk me through how can I accomplish it and meet the expected output with some originality if applicable according to it



Worked for 1m 21s >

I reviewed the complete MAD2_MP1_CampusClubConnect.pdf . This is similar to your Campus Companion exercise, but it must be built as a separate project from scratch.

Crucial requirements

The biggest rule

You must create a new Xcode project named:

Plain text



ClubConnect

Do not duplicate or rename your previous Campus Companion project. Reusing the concepts is allowed, but copying its project files will not satisfy the assignment.

Required screens

Your app needs two screens:

Screen	Required class	Purpose
Welcome screen	WelcomeViewController	Collects the student's information
Confirmation screen	ConfirmationViewController	Displays the submitted information

A UINavigationController must connect them, providing an automatic Back button.

Required welcome-screen elements

- App title, bold and at least 24 pt
- Subtitle/tagline
- Fixed-size, Aspect Fit image view
- Name text field with a placeholder
- Meeting-reminder switch
- Segmented control with Member and Officer
- Join button

The title must be connected as an IBOutlet and assigned inside viewDidLoad(), not merely hard-coded in the storyboard.

Required navigation and data passing

You must use:

- A manual Show segue drawn from the View Controller icon
- Segue identifier such as ShowConfirmationSegue

Work with ChatGPT



GPT-5.6 Terra Light





FEU INSTITUTE OF TECHNOLOGY

Development [AppDev2] ☁

Share ⋮ ⚙️ 📄

</> Plain text



Width: 100
Height: 100

This gives the Tamaraw Tech Guild an original technology-themed placeholder while satisfying the fixed width/height and Aspect Fit requirements.

Stop there and send a screenshot after adding the constraints.



Worked for 9s

The image, tint, Aspect Fit, top spacing, and horizontal centering are correct. However, the screenshot shows that **Width** and **Height** have not been added as constraints yet—the checkboxes beside them are still gray, and the button says **Add Constraints** but is disabled.

In the open **Add New Constraints** window:

1. Click the checkbox beside **Width**.
2. Click the checkbox beside **Height**.
3. Keep both values at:

</> Plain text



Width: 100
Height: 100

4. Click **Add 2 Constraints**.

Afterward, the Image View should have four constraints:

</> Plain text



Center X to Safe Area
Top Space to subtitle = 20
Width = 100
Height = 100

Then press **Command + B** and confirm that the build succeeds without layout warnings.



there are no layout warnings at all, checking the boxes just duplicates it so lets just ignore that



Worked for 39s

Work with ChatGPT



GPT-5.6 Terra Light ▾





FEU INSTITUTE OF TECHNOLOGY

Development [AppDev2] ☁

Share ...

📄 ⬆️ ↺️ 🔍 ...

done, is the last step about github



Worked for 16s

GitHub is the next major step, but not the absolute last. Since you changed the final design, first:

1. Press **Command + B** and confirm Build Succeeded.
2. Run both screens on the iPhone SE.
3. Confirm there are no yellow layout warnings.
4. Retake final screenshots because the previous ones do not include the new labels, card, and check icon.

Then commit and upload:

```
</> Bash
git status
git add -A
git commit -m "Complete ClubConnect app and final interface"
git push origin main
git status
```

After GitHub, the remaining submission tasks are:

- Add the 3–5 sentence reflection.
- Add an honest AI-use disclosure.
- Include the final screenshots.
- ZIP the complete project folder, including `.git`.
- Name it:

```
</> Plain text
Soriano_JohnRonen_MP1_ClubConnect.zip
```

Run the Git commands next and send me the resulting Terminal output.

📄 ⬆️ ↺️ 🔍 ...



it seems to work just fine

Worked for 18s

The final design looks polished and responsive on both devices. The labels, controls, confirmation icon, and message card all fit correctly.



Now commit it locally. In Terminal, go to the `ClubConnect` project folder and run:

Work with ChatGPT

+

GPT-5.6 Terra Light

